Internet das coisas

Segurança no Desenvolvimento de Software

Injection Attacks + Input Validation

Prof. Ana Beatriz Morais

AGO/2026

SENAI
PELO FUTURO DO TRABALHO

O que veremos hoje

- **Injection**
  - Como funciona uma injeção
  - Por que acontece
  - Como prevenir
- **Input validation**
  - Validação e sanitização
  - Blacklist vs Whitelist
- **Prática**



Injection Attacks

Injection Attacks (SQL, Command)

- **O que significa?**
  - Injection acontece quando dados fornecidos pelo usuário são interpretados pela aplicação como parte de uma instrução ou comando.
- **Exemplo**:
  - **DADO FORNECIDO**
  - Ana
  - **DADO INTERPRETADO COMO COMANDO**
  - ' OR '1'='1

Injection Attacks (SQL, Command)

- **A aplicação deve conseguir diferenciar:**
  - "DADO FORNECIDO"
  - de:
  - "INSTRUÇÃO"

Quando essa separação não existe, surge a vulnerabilidade.

Como funciona uma injeção?

- 👤 **USUÁRIO → Entrada**
  - **Ana Beatriz Morais ' OR '1'='1**
- 🖥️ **APLICAÇÃO →**
  - **"Vou colocar isso dentro da minha instrução"**
- 💾 **BANCO /** 🖥️ **SISTEMA →**
  - **FAZ A EXECUÇÃO**

Como funciona uma injeção?

- **Aplicação segura**
  1. 👤 **Usuário**
  2. 🔍 **Validação**
  3. 🧹 **Sanitização quando necessária**
  4. 🔒 **Parametrização**
  5. 💾 **Banco / Sistema**

Onde pode acontecer?

- Injection não acontece somente no SQL.
  - Pode acontecer em:
    - SQL Injection
    - Command Injection
    - NoSQL Injection
    - LDAP Injection
    - XSS
    - Template Injection

Por que Injection acontece?

- **Principais causas:**
  - Concatenação de strings.
  - Falta de parametrização.
  - Falta de validação de entrada.
  - Confiança excessiva nos dados enviados pelo cliente.
  - Execução direta de comandos.
  - Falta de tratamento adequado das entradas.
  - Código desenvolvido sem considerar segurança.

O CLIENTE NÃO É CONFIÁVEL

Se eu criei um formulário com React, o usuário só consegue enviar o que está no formulário?

SIM OU NÃO?

O CLIENTE NÃO É CONFIÁVEL

- **O usuário pode:**
  - **Alterar o HTML**
  - **Alterar JavaScript**
  - **Utilizar Postman**
  - **Utilizar Insomnia**
  - **Utilizar curl**
  - **Criar sua própria requisição HTTP**
  - **Alterar parâmetros**
  - **Ignorar completamente o front-end**

Ataque na prática

Onde está a vulnerabilidade deste código?

```
app.get("/usuario", async (req, res) => {

    const nome = req.query.nome;

    const sql = `
        SELECT *
        FROM usuarios
        WHERE nome = '${nome}'
    `;

    const [usuarios] = await db.query(sql);

    res.json(usuarios);
});
```

Ataque na prática

Onde está a vulnerabilidade deste código?

‘${nome}’

O valor recebido pelo usuário está sendo colocado diretamente dentro da consulta.

```
app.get("/usuario", async (req, res) => {

    const nome = req.query.nome;

    const sql = `
        SELECT *
        FROM usuarios
        WHERE nome = '${nome}'
    `;

    const [usuarios] = await db.query(sql);

    res.json(usuarios);
});
```

Como corrigir?

❌ Vulnerável

JavaScript

```
const sql = `
    SELECT *
    FROM usuarios
    WHERE nome = '${nome}'
`;
```

✅ Parametrizado

JavaScript

```
const sql = `
    SELECT *
    FROM usuarios
    WHERE nome = ?
`;

const [usuarios] = await db.execute(sql, [nome]);
```

Como prevenir?

Quatro camadas de proteção

1. **VALIDAR**
    a. Verifica se o dado está dentro do que é permitido
2. **SANITIZAR (quando necessário)**
    a. Remove ou transforma elementos indesejados quando isso fizer sentido
3. **PARAMETRIZAR**
    a. Separa os dados das instruções/comandos
4. **LIMITAR PERMISSÕES**
    a. Reduz o que uma conta/processo pode fazer caso algo seja comprometido

Input validation

Input validation

- **É verificar se o dado recebido:**
  - É do tipo esperado?
  - Está no formato esperado?
  - Está dentro do tamanho permitido?
  - Está dentro dos valores permitidos?

Input validation

- **Exemplo**
  - **Campo IDADE (inteiro)**
    - **Esperado: 18 ou 10 ou 29**
  - **Recebido**
    - **“banana”**

Inválido

Validar tudo que vem do usuário

Entradas que precisam ser consideradas não confiáveis:

- **req.body**
- **req.params**
- **req.query**
- **Headers**
- **Cookies**
- **Uploads**
- **JSON**
- **Formulários**

FRONT-END x BACK-END

- **FRONT-END**
  - Melhora experiência do usuário
  - Feedback mais rápido
- **BACK-END**
  - Valida novamente
  - Protege o sistema

Exemplo de validação no back-end

O que está sendo feito?

1. Recebendo os dados
2. Validando se realmente é o dado que o sistema precisa
3. Dependendo da resposta o sistema aceita ou rejeita

```
app.post("/usuarios", (req, res) => {

    const { nome, idade, email } = req.body;

    if (!nome || typeof nome !== "string") {
        return res.status(400).json({
            erro: "Nome inválido"
        });
    }

    if (!Number.isInteger(idade) || idade < 0) {
        return res.status(400).json({
            erro: "Idade inválida"
        });
    }

});
```

Blacklist

Você define o que NÃO PODE.

- **Não pode:**
  - script
  - comando X
  - carácter Y
  - palavra Z
- **Problema: O atacante pode encontrar outra forma de escrever a mesma coisa.**

Whitelist / Allowlist

Você define o que PODE.

- **Exemplo**
  - Campo ESTADO → Valores permitidos
    - SC, RS, MG, SP, GO
    - Se receber XX
    - SISTEMA REJEITA
- **A pergunta é: “Isso é permitido?”**

Validação deve ser específica

- **Campo CPF**
  - **Formato**
  - **Dígitos**
  - **Regras**
- **Campo E-MAIL**
  - **formato esperado**
- **Campo IDADE**
  - **numero + faixa permitida**

Validação deve ser específica

Qual erro deste código?

```
app.post("/usuarios", (req, res) => {

    const { nome, idade, estado } = req.body;

    salvarUsuario(nome, idade, estado);

    res.json({
        sucesso: true
    });
});
```

Validação deve ser específica

- **Qual erro deste código?**
  - **NENHUM DESSES CAMPOS FORAM VALIDADOS.**

```
app.post("/usuarios", (req, res) => {

    const { nome, idade, estado } = req.body;

    salvarUsuario(nome, idade, estado);

    res.json({
        sucesso: true
    });
});
```

Validação deve ser específica

- **O formulário espera:**
  - { “idade”: 29}
- **Mas o usuario envia:**
  - { “idade”: “vinte e nove” }
  - { “idade”: -500 }
  - { “idade”: 99999 }
- **Ser do tipo correto significa necessariamente ser válido?**
  - **Não**

Prática: Validação e Parametrização de dados

SENAI

Serviço Nacional de Aprendizagem Industrial

PELO FUTURO DO TRABALHO

0800 048 1212 **sc.senai.br**

Rodovia Admar Gonzaga, 2765 - Itacorubi - 88034-001 - Florianópolis, SC